

Lab11 – RBAC ServiceAccount

In this exercise, you will define Role Based Access Control (RBAC) to grant permissions to a service account. The permissions should only apply to certain API resources and operations.

1. Create a new namespace named `t23`. Create the service account `api-call` in the namespace.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl create namespace t23
namespace/t23 created
vagrant@kubernetes-control-plane:~/lab$ kubectl get ns
NAME                STATUS   AGE
default             Active   23h
kube-node-lease     Active   23h
kube-public         Active   23h
kube-system         Active   23h
t23                 Active   4s
vagrant@kubernetes-control-plane:~/lab$
```

```
vagrant@kubernetes-control-plane:~/lab$ kubectl create serviceaccount api-call -n t23
serviceaccount/api-call created
vagrant@kubernetes-control-plane:~/lab$ kubectl get sa -n t23
NAME      SECRETS   AGE
api-call  0         11s
default   0         12m
vagrant@kubernetes-control-plane:~/lab$
```

2. Create a Pod named `service-list` in the namespace `t23`. The container uses the image `alpine/curl:3.14` and makes a `curl` call to the Kubernetes API that lists Service objects in the `default` namespace in an infinite loop. Attach the service account `api-call` to the Pod.

```
vagrant@kubernetes-control-plane:~/lab$ vim pod.yaml
vagrant@kubernetes-control-plane:~/lab$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: service-list
  namespace: t23
spec:
  # serviceAccountName: api-call
  containers:
  - name: service-list
    image: alpine/curl:3.14
    command: ['sh', '-c', 'while true; do curl -s -k -m 5 -H "Authorization: Bearer $(cat /var/run/secrets/kubernetes.io/serviceaccount/token)" https://kubernetes.default.svc.cluster.local/api/v1/namespaces/default/services; sleep 10; done']
vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f pod.yaml
pod/service-list created
vagrant@kubernetes-control-plane:~/lab$
```

- Inspect the container logs after the Pod has been started. What response do you expect to see from the ``curl`` command?

```
vagrant@k8s-control-plane:~/lab$ kubectl get pod -n t23
NAME          READY   STATUS    RESTARTS   AGE
service-list  1/1     Running   0           107s
vagrant@k8s-control-plane:~/lab$ kubectl logs service-list -n t23
{"kind": "Status",
 "apiVersion": "v1",
 "metadata": {},
 "status": "Failure",
 "message": "services is forbidden: User \"system:serviceaccount:t23:default\" cannot list resource \"services\" in API group \"\" in the namespace \"default\"",
 "reason": "Forbidden",
 "details": {
  "kind": "services"
 },
 "code": 403
}{
 "kind": "Status",
 "apiVersion": "v1",
 "metadata": {},
 "status": "Failure",
 "message": "services is forbidden: User \"system:serviceaccount:t23:default\" cannot list resource \"services\" in API group \"\" in the namespace \"default\""
```

- Assign a ClusterRole and RoleBinding to the service account that only allows the operation needed by the Pod. Have a look at the response from the ``curl`` command.

```
vagrant@k8s-control-plane:~/lab$ vim clusterrole.yaml
vagrant@k8s-control-plane:~/lab$ cat clusterrole.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: list-services-clusterrole
rules:
- apiGroups: [""]
  resources: ["services"]
  verbs: ["list"]
vagrant@k8s-control-plane:~/lab$ vim rolebinding.yaml
vagrant@k8s-control-plane:~/lab$ cat rolebinding.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: serviceaccount-service-rolebinding
subjects:
- kind: ServiceAccount
  name: api-call
  namespace: t23
roleRef:
  kind: ClusterRole
  name: list-services-clusterrole
  apiGroup: rbac.authorization.k8s.io
vagrant@k8s-control-plane:~/lab$ kubectl apply -f clusterrole.yaml
clusterrole.rbac.authorization.k8s.io/list-services-clusterrole created
vagrant@k8s-control-plane:~/lab$ kubectl apply -f rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/serviceaccount-service-rolebinding created
vagrant@k8s-control-plane:~/lab$
```

The API call running inside of the container should now be authorized and be allowed to list the Service objects in the ``default`` namespace. As shown in the output below, the namespace currently hosts at least one Service object, the ``kubernetes.default`` Service.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl logs service-list -n t23
{
  "kind": "ServiceList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "62257"
  },
  "items": [
    {
      "metadata": {
        "name": "kubernetes",
        "namespace": "default",
        "uid": "e1d22135-427f-473d-842c-e1c2ac0155f7",
        "resourceVersion": "43933",
        "creationTimestamp": "2023-08-12T15:02:44Z",
        "labels": {
          "component": "apiserver",
          "provider": "kubernetes"
        }
      },
      "managedFields": [
        {
          "manager": "kube-apiserver",
          "operation": "Update",
          "apiVersion": "v1",
          "time": "2023-08-12T15:02:44Z",
          "fieldsType": "FieldsV1",
          "fieldsV1": {
            "f:metadata": {
              "f:labels": {
                ".": {},
                "f:component": {},
                "f:provider": {}
              }
            }
          }
        }
      ]
    }
  ]
}
```